

El proyecto trata de definir una arquitectura de ciberseguridad, con el diseño y el análisis de sus funcionalidades. El proyecto tendrá 3 fases (ideación, desarrollo y diseño, análisis y conclusión).

Requerimientos

R1. Los grupos serán de 2 a 4 participantes. En caso de grupos de 1 o 5 participantes se necesitará justificación y se tendrá en valoración en base a las ODS.

R2. El proyecto tiene que definir distintas estructuras de datos y tiene que demostrar trabajar simultáneamente con distintas complejidades y/o recursividad.

R3. El proyecto requiere programación orientada a objetos (POO) y patrones (polimorfismo)

R4. Se requiere que todos los integrantes commiten en el mismo Git

R5. Se permiten utilizar herramientas y librerías externas tanto para el desarrollo de código como para la generación de diagramas y análisis de los algoritmos.

R6. No se permite utilizar LLMs y chatbots para desarrollar la mayoría de parte funcional del proyecto. En el caso de utilizarse para proponer soluciones o resolver problemas se debe mencionar explícitamente en cada presentación y en comentarios en el código.

Fase 1 - Ideación del proyecto [Entrega: Presentación en PDF]

En esta fase hay que presentar el proyecto con los siguientes contenidos:

- Introducción a vuestro sistema de ciberseguridad
- Problemática a resolver y estado del arte (sistemas existentes y competidores)
- Estructuras de datos y herramientas (librerías) requeridas
- Listado de funcionalidades
- Innovación y proceso de escalado (cálculo de complejidad preliminar)

Fase 2 - Desarrollo y Diseño del sistema [Entrega: ZIP/RAR de código y archivos]

En esta fase hay que desarrollar y diseñar la arquitectura del proyecto:

- Desarrollo de código preliminar en python
- Diseño de la POO con diagramas UML
- Diagramas de flujo de cada una de las funcionalidades/funciones
- Cálculo de complejidad y costes de las funciones (peor caso)

Fase 3 - Análisis y Conclusión [Entrega: Presentación en PDF y ZIP/RAR con archivos]

En esta fase, se actualizará el desarrollo presentado en la fase 2, se analizarán los algoritmos y hará una presentación final.

- Propuesta de nuevas funcionalidades
- Análisis de tiempos promedios
- Análisis de complejidad y costes en mejor/promedio/peor caso
- (Opcional) Actualización del código
- (Opcional) Actualización de los diagramas o diagramas de nuevas funcionalidades
- Conclusión: propuestas a futuro y optimización
- Presentación final





Ejemplos de casos de uso en sistemas de ciberseguridad

E1. Sistema de Detección de Intrusos (IDS) Basado en Reglas y Patrones Este sistema analiza el tráfico de red simulado para detectar anomalías o ataques conocidos. Utilizaría **Colas (Queues)** para procesar los paquetes de red entrantes y **Árboles Trie** o grafos para buscar firmas de ataques maliciosos de forma eficiente. La recursividad se aplicaría en el recorrido profundo de estos árboles para emparejar patrones complejos. A nivel de P00, podrías tener una clase base Analizador con polimorfismo en clases hijas como AnalizadorDDoS, AnalizadorInyeccionSQL o AnalizadorMalware, cada una con su propia implementación del método evaluarPaquete().

E2. Simulador de Gestión de Identidades y Accesos (IAM) con RBAC El objetivo es diseñar un sistema que gestione qué usuarios tienen acceso a qué recursos dentro de una empresa. El estado del arte incluye sistemas como Active Directory. Utilizaría **Árboles y Grafos** para representar la jerarquía de roles (Role-Based Access Control) y los permisos heredados. La recursividad es perfecta aquí para calcular si un usuario tiene acceso a un archivo verificando sus roles padre. La P00 se implementaría con clases como Entidad, extendida polimórficamente a Usuario, Grupo y Recurso, interactuando según tipo.

E3. Motor de Análisis y Clasificación de Malware por Firmas Un antivirus básico que escanea un sistema de archivos simulado. Utilizaría **Árboles (o grafos dirigidos)** para representar el sistema de directorios del ordenador, usando recursividad para navegar y escanear cada subcarpeta y archivo. Emplearía **Tablas Hash (Diccionarios)** para almacenar y consultar rápidamente millones de firmas de virus (hashes MD5/SHA256). La P00 brillaría con una clase abstracta EstrategiaEscaneo, implementando polimorfismo: EscaneoRapido y EscaneoProfundo.

E4. Gestor y Analizador de Redes Honeybot (Señuelos de Red) Un sistema que simula una red de servidores vulnerables para atrapar atacantes y registrar sus movimientos. Emplearía **Grafos** para trazar la topología de la red falsa y los saltos que da el atacante (movimiento lateral). Se usarían algoritmos recursivos para rastrear el origen de una intrusión a través de múltiples nodos. Usarías P00 para definir una clase genérica NodoSeñuelo, de la cual heredarían polimórficamente ServidorFTPFalso, BaseDeDatosFalsa y ServidorWebFalso, cada uno respondiendo de forma diferente a los comandos del atacante.

E5. Escáner de Vulnerabilidades Web (Web Crawler de Seguridad) Una herramienta que escanea un sitio web en busca de enlaces rotos, puertos abiertos o vulnerabilidades (como falta de cabeceras de seguridad). Utilizaría un **Grafo** donde cada nodo es una URL y cada arista un enlace hipertexto. La recursividad (como Búsqueda en Profundidad - DFS) es esencial para navegar de un enlace a otro hasta mapear toda la web. A nivel de P00, podrías tener una interfaz DetectorVulnerabilidad con implementaciones polimórficas como DetectorXSS y DetectorSQLi que se ejecutan sobre el código HTML recolectado.



E6. Analizador de Rutas Seguras (Enrutamiento Onion / VPN Multicapa) Un simulador de cómo los datos viajan a través de nodos encriptados (estilo Tor). Requeriría un **Grafo ponderado** para representar los nodos de la red global y sus latencias/niveles de confianza. Utilizarías algoritmos de grafos (como Dijkstra o A*) para encontrar la ruta más rápida y segura, usando recursividad para la resolución de caminos alternativos si un nodo cae. La POO se usaría para crear nodos con diferentes capacidades criptográficas, con una clase Cifrador y clases polimórficas CifradorAES y CifradorRSA.

E7. Sistema de Auditoría de Contraseñas (Cracker Ético) Una herramienta que evalúa la fortaleza de las contraseñas de una base de datos filtrada. Se apoyarían en **Árboles de búsqueda** para diccionarios de palabras y recursividad para generar permutaciones y combinaciones (fuerza bruta inteligente con variaciones de caracteres). Usarías POO creando una clase base AtaqueContraseña con métodos polimórficos como ejecutarAtaque() que se comportarán distinto en subclases como AtaqueDiccionario, AtaqueFuerzaBruta o AtaqueHibrido.

E8. Registro de Auditoría Inmutable (Audit Log basado en Merkle Trees) Ideal para evitar que un atacante borre sus huellas en los logs del sistema. Construirías un **Árbol de Merkle (Merkle Tree)**, una estructura de datos basada en árboles binarios y hashes criptográficos. La recursividad es el estándar para recalculando los hashes desde las hojas (los logs) hasta la raíz (el hash principal). Usarías POO para definir los Nodos del árbol y diferentes tipos de EventosDeLog (e.g., EventoLogin, EventoBorradoArchivo) que sobrescriben polimórficamente su forma de serializarse antes de ser hasheados.

E9. Motor de Mitigación de Ataques DDoS Un sistema que evalúa el tráfico en tiempo real y descarta conexiones maliciosas antes de que saturan el servidor. Usarías **Colas de Prioridad (Priority Queues)** para gestionar el ancho de banda y **Tablas Hash** para llevar la cuenta de las peticiones por IP por segundo. La recursividad podría usarse en la evaluación de subredes bloqueadas. Con la POO, diseñarías una clase abstracta ReglaFiltrado con implementaciones polimórficas como FiltroGeolocalizacion, FiltroFrecuenciaIP o FiltroAnomaliasHTTP.

E10. Filtro Antispam y Detección de Phishing Semántico Un sistema que protege la bandeja de entrada leyendo la estructura y el contenido del correo electrónico. Utilizaría **Árboles de Decisión** para clasificar si un correo es spam o no basándose en características extraídas. Emplearía recursividad para analizar correos que contienen hilos de mensajes anidados o múltiples archivos adjuntos. La POO incluiría una clase AnalizadorContenido con polimorfismo para analizar diferentes partes del correo (clases AnalizadorAsunto, AnalizadorCuerpoTexto, AnalizadorCabeceras).



E11. Detector de URLs Engañosas (Typosquatting) por Distancia de Textos Este proyecto detecta si una URL intenta suplantar a una web legítima (como "g00gle.com" en lugar de "google.com") buscando patrones de similitud. Para las estructuras de datos, usarían **Tablas Hash (Diccionarios)** para almacenar la lista de dominios legítimos permitidos y acceder a ellos instantáneamente. La **recursividad** brillaría al implementar algoritmos clásicos de comparación de cadenas, como el cálculo de la Distancia de Levenshtein, que de forma natural se programa recursivamente para ver cuántos cambios separan una palabra de otra. En cuanto a la **P00 y polimorfismo**, podrían crear una interfaz o clase abstracta llamada **MetricaSimilitud**, de la cual heredarían clases como **MetricaLevenshtein**, **MetricaHamming** o **MetricaCoincidenciaParcial**, cada una implementando el método **calcularDistancia()** a su manera.

E12. Clasificador Básico de Correos Spam/Phishing (Frecuencia de Palabras) Un filtro que analiza el texto de un correo y decide si es peligroso basándose en la repetición de palabras clave (como "urgente", "contraseña", "premio"). El sistema utilizaría **Árboles Binarios de Búsqueda (BST)** o **Diccionarios** para mapear cada palabra con su frecuencia de aparición. La **recursividad** se utilizaría para explorar los directorios del disco duro (simulando una bandeja de entrada con subcarpetas) e ir leyendo archivo por archivo, o para recorrer el árbol BST y calcular un puntaje de riesgo total. La **P00** se demostraría con una clase principal **FiltroCorreo** que usa **polimorfismo** en subclases como **FiltroPorPalabrasClave**, **FiltroPorRemitenteSospechoso** y **FiltroPorEnlacesOcultos**.

E13. Escáner de Firmas de Malware con Árboles Trie Un antivirus simplificado que busca secuencias de bytes maliciosas (patrones) dentro de archivos de texto o código simulado. La estructura de datos estrella aquí es el **Árbol Trie (Árbol de Prefijos)**, que es perfecto (y muy propio de 2º año) para almacenar miles de firmas de virus letra por letra o byte por byte. Tanto la inserción de las firmas en el árbol como la búsqueda del patrón dentro del archivo se resuelven mediante funciones **recursivas** muy elegantes. Para la **P00**, diseñarían una clase **MotorEscaneo** con métodos **polimórficos** implementados en **EscaneoFirmaExacta** (busca el patrón tal cual) y **EscaneoFirmaParcial** (permite saltarse algunos caracteres simulando malware mutado).

E14. Detección de Fraude con Análisis de Ciclos en Grafos Este sistema analiza transacciones de dinero entre usuarios para detectar patrones de blanqueo o fraude básico, como el dinero que da vueltas entre varias cuentas para ocultar su rastro. Los estudiantes programarían un **Grafo Dirigido**, donde los nodos son cuentas y las flechas son las transferencias. La **recursividad** se aplicaría usando el clásico algoritmo de Búsqueda en Profundidad (DFS) para encontrar "ciclos" en el grafo (por ejemplo, A paga a B, B a C, y C devuelve a A). El requerimiento de **P00** se cubriría con una clase abstracta **ReglaFraude** y **polimorfismo** en clases como **ReglaCicloCerrado** (detecta los anillos), **ReglaMontoAnomalo** (detecta picos de dinero) y **ReglaCuentasNuevas**.

E15. Bot de Detección de Ataques de Fuerza Bruta en Logs Un analizador que lee un archivo de registro (log) de un servidor y detecta patrones que indiquen que alguien está intentando adivinar una contraseña. Se utilizarían **Colas (Queues)** para almacenar los últimos intentos de conexión en una ventana de tiempo (ej. últimos 5 minutos). La **recursividad** podría aplicarse en una función que analice secuencias repetitivas dentro de esos logs o que agrupe IPs por subredes de forma jerárquica. La **P00** se implementaría definiendo un **AnalizadorEvento** con comportamiento **polimórfico** en clases hijas como **AnalizadorLoginFallido**, **AnalizadorEscaneoPuertos** y **AnalizadorPeticionesWeb**.



E16. Motor de Árbol de Decisión para Identificar Perfiles Falsos (Bots) Un sistema de IA clásica donde los estudiantes construyen su propio "Árbol de Decisión" programable para clasificar si un usuario de red social es real o un bot (basado en ratio de seguidores, frecuencia de mensajes, etc.). La estructura de datos es literalmente un **Árbol Binario** personalizado. La **recursividad** es indispensable aquí para recorrer el árbol: el sistema evalúa un nodo (ej. "¿tiene foto de perfil?") y recursivamente llama al hijo izquierdo o derecho hasta llegar a una conclusión (hoja). La **P00** brillaría al definir una clase abstracta **NodoArbol**, de la cual heredarían polimórficamente **NodoCondicion** (que evalúa una regla y sigue bajando) y **NodoResultado** (que devuelve si es Bot o Humano).

E17. Simulador de Rutas de Red Seguras (Detección de Nodos Caídos) Un programa que calcula cómo enviar paquetes de datos esquivando servidores (nodos) que han sido clasificados como "comprometidos" o hackeados. Esto requiere implementar un **Grafo Ponderado**, algo fundamental en 2º año. La **recursividad** se usaría en la implementación del algoritmo de Dijkstra, o más simplemente, en un backtracking recursivo que encuentre todos los caminos posibles entre el punto A y el B esquivando los nodos malos. La **P00** y **polimorfismo** se justificarían con una interfaz **EstrategiaRuta** implementada por **RutaMasCorta** (prioriza pocos saltos) y **RutaMasSegura** (prioriza nodos con alto nivel de confianza).

E18. Auditor de Permisos y Roles (RBAC) con Herencia Un sistema que detecta anomalías de acceso, comprobando si un empleado está intentando acceder a carpetas que no le corresponden. El sistema de permisos se modelaría con **Árboles N-arios**, donde el directorio raíz (ej. "Empresa") se divide en departamentos y subcarpetas, cada uno con niveles de permisos. La **recursividad** es la forma natural de verificar si un usuario tiene acceso: si no tiene permiso explícito en un archivo, la función busca recursivamente si lo heredó de la carpeta padre. La **P00** incluiría clases como **EntidadAcceso**, con polimorfismo en sus subclases **UsuarioNormal**, **Administrador** y **AuditorInvitado**, reaccionando distinto al método **solicitarAcceso()**.

E19. Generador Predictivo de Contraseñas (Ataque de Diccionario Inteligente) Una herramienta defensiva que evalúa la seguridad de las contraseñas de los usuarios simulando cómo las adivinaría un hacker. Utilizarían **Listas** y **Tablas Hash** para cargar diccionarios de palabras comunes. La **recursividad** sería la protagonista mediante un algoritmo de *backtracking* que genere combinaciones y permutaciones de palabras con números (ej. "admin" -> "admin123", "@dmin", etc.) para intentar romper el hash de la contraseña. A nivel de **P00**, tendrían una clase base **AtaquePassword** con métodos polimórficos en las clases **AtaqueFuerzaBrutaPura**, **AtaqueDiccionario** y **AtaqueSustitucionCaracteres**.

E20. Honeypot Simulado Basado en Máquina de Estados Un programa interactivo que finge ser un servidor Linux vulnerable. El atacante (usuario) introduce comandos, y el sistema detecta su patrón de ataque para darle respuestas falsas. Esto se modela mediante un **Grafo Dirigido**, que actúa como una máquina de estados finitos (cada comando del atacante lo mueve a un estado diferente del engaño). La **recursividad** se puede usar para validar si la secuencia de comandos históricos del atacante coincide con un patrón de ataque conocido guardado en una estructura anidada. La **P00** se aplica con una interfaz **ComandoHoneypot** y clases polimórficas como **ComandoNavegacion** (para simular "cd" o "ls") y **ComandoEjecucion** (para simular "chmod" o "./script").

